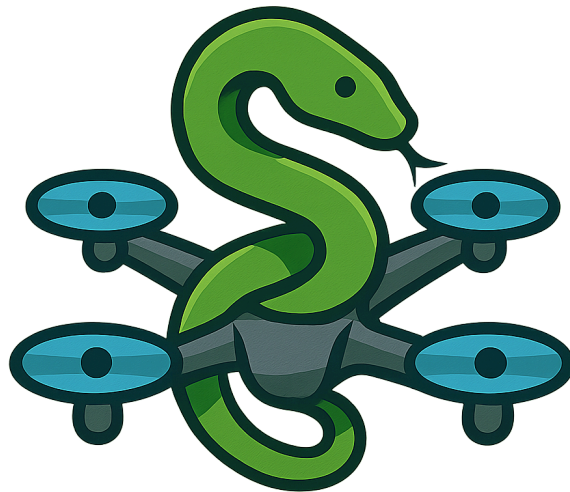


Technische Hochschule Ostwestfalen-Lippe

Bachelor of Science, Data Science

Gamification



Ausarbeitung zum Thema:

Vom Prototyp zur Spielschleife

Gamifizierung von *PythonWithDrones*

Erstellt von:	Niclas Falke, Stefan Reitemeyer und Jonas Pottmeier
Matrikelnummer:	20122542 (Niclas), 20124913 (Stefan) und 20116026 (Jonas)
Fachsemester:	4
Abgabedatum:	28.09.2026
Prüfer:	Prof. Dr.-Ing. Rainer Rasche

Inhaltsverzeichnis

Abstrakt	1
1 Einleitung	1
2 Stand der Forschung	1
3 Ausgangslage	2
3.1 Der Stand zum Modulwechsel	2
3.2 Grenzen des Ausgangsstands	2
4 Begriffe und Festlegungen	3
4.1 Gamifizierung	3
4.2 Mechanik, Dynamik und Erfahrung	3
4.3 Drei Grundbedürfnisse	3
4.4 Passung von Anforderung und Fähigkeit	3
4.5 Festlegungen für jedes Spielelement	4
5 Methodik	4
5.1 Überblick	4
6 Ergebnisse	4
6.1 Erweiterter Handlungsraum und neue Bausteine	5
6.1.1 Neue Befehle	5
6.1.2 Lösungsbedingungen	6
6.2 Aufgabenkurve	6
6.3 Auswertung eines Laufs	7
6.3.1 Kennzahlen	7
6.3.2 Abschlussanzeige	7
6.3.3 Terminal	8
6.4 Bestenliste	8
6.4.1 Architektur	8
6.4.2 Was gewertet wird	9
6.4.3 Betrieb ohne Backend	10
6.5 Level-Editor	11
6.5.1 Aufbau	11
6.5.2 Vom Entwurf zum Level	11
6.6 Reibungsabbau	12
7 Diskussion	13
7.1 Zuordnung zu den Grundbedürfnissen	13
7.2 Einschränkungen und Reichweite der Aussagen	13
7.3 Bewusst getroffene Festlegungen	14
7.4 Wertungslogik und Motivation	14
8 Schlussfolgerung	15
8.1 Zusammenfassung	15
8.2 Nächste Schritte	15
Literatur	16

Abstrakt

PythonWithDrones ist eine Lernanwendung, in der Python-Quelltext eine virtuelle Drohne durch dreidimensionale Level steuert. Zum Abschluss des Vorgängermoduls lag ein lauffähiger Prototyp vor, dessen Ablauf mit dem Erreichen des Zielportals endete [1]. Danach war die Interaktion beendet. Die vorliegende Arbeit beschreibt, wie diese Anwendung zwischen dem 10. April und dem 28. September 2026 zu einem Spiel mit geschlossener Schleife ausgebaut wurde. Hinzugekommen sind ein erweiterter Handlungsraum der Drohne mit Schieben, Aufnehmen, Abliefern und Vorausschauen, eine auf dreizehn Level verlängerte Aufgabenkurve, eine Auswertung jedes Laufs nach Abstürzen, geflogenen Feldern und Codezeilen, eine serverseitige Bestenliste mit Kontoverwaltung und Versuchsverlauf sowie ein dreidimensionaler Level-Editor, mit dem Spielende eigene Aufgaben bauen und einreichen können. Die Auswertung ordnet die umgesetzten Elemente den drei Grundbedürfnissen Kompetenz, Autonomie und soziale Eingebundenheit zu und begründet, wie die einzelnen Entwurfsentscheidungen zusammenwirken.

1 Einleitung

Zum Abschluss des vorangegangenen Moduls lag eine funktionsfähige Anwendung vor. Python wurde im Browser ausgeführt, die Drohne ließ sich steuern, und die vorhandenen Level waren lösbar. Die Anwendung unterschied je Level jedoch nur zwei Zustände, das Erreichen oder das Nichterreichen des Zielportals. Der Umfang einer Lösung, die Zahl der Fehlversuche und die Frage, ob nach Abschluss des letzten Levels ein Anlass zur weiteren Nutzung bestand, blieben unberücksichtigt.

Die vorliegende Arbeit hat das Ziel, diese Anwendung um Spielelemente zu erweitern und die dabei getroffenen Entwurfsentscheidungen zu begründen. Untersucht wird, welche Elemente aus dem Spieldesign sich in eine bestehende Lernanwendung einfügen lassen, welche Wirkung strukturell von ihnen zu erwarten ist und welche Kosten sie im Betrieb verursachen. Die Ausführung von Python im Browser ist nicht Gegenstand der Arbeit, da sie im Vorgängermodul entwickelt wurde [1]. Gamifizierung wird nicht als nachträgliche Ergänzung einer fertigen Anwendung verstanden, sondern als Eingriff in deren Regeln, der verändert, worauf Lernende beim Schreiben ihres Programms achten.

Die Arbeit stützt sich auf den Quelltext des Projekts und seine Versionsgeschichte [2] sowie auf die Ausarbeitung des Vorgängermoduls [1]. Als Ausgangspunkt dient das Tag `v1.0.0` vom 10. April 2026, das den Stand zum Abschluss des Vorgängermoduls markiert. Der Betrachtungszeitraum umfasst die Änderungen zwischen diesem Stand und dem 28. September 2026.

2 Stand der Forschung

Die Umsetzung baut auf einem etablierten Satz an Werkzeugen aus dem Web- und Backend-Bereich auf, ohne selbst neue Technologie einzuführen. Next.js dient als React-Framework für den statischen Seitenexport [3]. Pyodide bringt einen zu WebAssembly übersetzten CPython-Interpreter in den Browser und macht damit eine serverseitige

Python-Ausführung überflüssig [4]. React Three Fiber bindet die 3D-Bibliothek Three.js deklarativ in React-Komponenten ein und trägt die im Browser laufende Drohnenszene [5]. Auf der Serverseite stellt FastAPI die asynchrone Web-Schnittstelle der Bestenliste bereit [6], SQLAlchemy vermittelt den objektrelationalen Zugriff darauf [7], und PostgreSQL hält die Daten persistent [8]. Alle fünf Bausteine sind quelloffen, gut dokumentiert und bereits im Vorgängermodul beziehungsweise im bestehenden Projekt im Einsatz [1]. Die vorliegende Arbeit erweitert diesen Stack um die in Abschnitt 3 beschriebenen Fähigkeiten, statt ihn auszutauschen.

3 Ausgangslage

3.1 Der Stand zum Modulwechsel

Die Anwendung ist ein statischer Export eines Next.js-Projekts [3] und wird über GitHub Pages ausgeliefert. Sie benötigte zu diesem Zeitpunkt keine serverseitige Komponente. Geschriebener Quelltext wird an einen Web Worker übergeben, in dem Pyodide einen zu WebAssembly übersetzten CPython-Interpreter betreibt [4]. Jeder Drohnenbefehl erzeugt dort ein Datenobjekt, das über `postMessage` an den Hauptthread zurückgereicht wird. Dort nimmt eine Warteschlange die Objekte entgegen und arbeitet sie nacheinander als Animationen in einer mit React Three Fiber aufgebauten Szene ab [5]. Der Fortschritt liegt im `localStorage` des Browsers. Der Aufbau dieses Prototyps ist in der Ausarbeitung des Vorgängermoduls ausführlich beschrieben [1].

Level sind YAML-Dateien. Sie beschreiben den Startpunkt der Drohne und eine Folge horizontaler Schichten, in denen jede Zelle über eine Block-Kennung belegt ist. Welche Eigenschaften eine solche Kennung hat, ob ein Block also eine Kollision auslöst oder als Ziel gilt, steht zentral in der Blockregistrierung. Sowohl die Python-Seite als auch die Darstellung greifen auf dieselbe Quelle zu.

Zum Zeitpunkt von `v1.0.0` umfasste dieser Aufbau sechs spielbare Level, neun Blocktypen und zehn öffentliche Methoden am Drohnenobjekt. Der Handlungsraum bestand aus Bewegen, Steigen, Sinken, Drehen und zwei Abfragen, mit denen sich ein blockierter Weg und das Erreichen des Portals prüfen ließen.

3.2 Grenzen des Ausgangsstands

Aus der Perspektive des Gamification-Moduls fehlten dem beschriebenen Aufbau drei Voraussetzungen. Zunächst gab es keine Bewertung einer Lösung. Die Anwendung erfasste allein deren Vorhandensein und bot damit keine Grundlage für eine Rückmeldung, die über die Feststellung des Erfolgs hinausgeht, und keinen Maßstab für Verbesserung. Weiter gab es keinen Bezug zu anderen Nutzenden, da der Fortschritt ausschließlich lokal im Browser abgelegt war. Zwei Personen, die dasselbe Level lösten, erfuhren nichts voneinander, und ein Gerätewechsel verwarf den Fortschritt. Schließlich gab es keinen Spielraum für eigene Inhalte. Level lagen als YAML-Dateien im Repository, und ein eigener Beitrag setzte Kenntnis der Dateistruktur, ein Auschecken des Projekts und einen Pull Request voraus, was für die Zielgruppe keine erreichbare Handlung darstellt.

4 Begriffe und Festlegungen

Dieser Abschnitt legt fest, in welcher Bedeutung die zentralen Begriffe im weiteren Verlauf verwendet werden und woran die Entwurfsentscheidungen anschließend gemessen werden.

4.1 Gamifizierung

Gamifizierung bezeichnet in dieser Arbeit den Einsatz einzelner Elemente aus dem Spieldesign in einem Zusammenhang, der selbst kein Spiel ist [9]. Maßgeblich ist die Beschränkung auf einzelne Elemente. Die Lernanwendung wird nicht zu einem Spiel umgebaut. Übernommen werden stattdessen einzelne Bausteine, deren Wirkung aus Spielen bekannt ist. Die drei am häufigsten übernommenen Bausteine sind Punkte, Abzeichen und Bestenlisten [10]. Punkte machen eine Leistung zählbar, Abzeichen machen sie sichtbar, und Bestenlisten machen sie vergleichbar. Im vorliegenden Projekt sind die Punkte als Kennzahlen je Lauf und die Bestenliste umgesetzt, während Abzeichen zurückgestellt sind.

4.2 Mechanik, Dynamik und Erfahrung

Der Entwurf trennt drei Ebenen, wie sie im MDA-Rahmenwerk des Spieldesigns beschrieben sind [11]. Die Mechanik ist die Regel, die im Quelltext geschrieben wird. Die Dynamik ist das Verhalten, das im Betrieb daraus entsteht. Die Erfahrung ist das Ergebnis, das bei den Spielenden ankommt. Die Trennung ist nützlich, weil auf der Ebene der Mechanik gebaut wird, während die Wirkung auf der Ebene der Erfahrung eintritt und die dazwischenliegende Dynamik sich nur mittelbar festlegen lässt.

Im vorliegenden Projekt lässt sich das an der Bestenliste zeigen. Die Mechanik ist deren Sortierreihenfolge. Die daraus entstehende Dynamik ist, dass Spielende ihren Lösungsweg kürzen, statt ihn nur lauffähig zu machen. Die Erfahrung ist die eines gefundenen, kompakten Lösungswegs. Die Aufgabe des Entwurfs besteht darin, die Mechanik so zu wählen, dass die beabsichtigte Dynamik entstehen kann.

4.3 Drei Grundbedürfnisse

Als Ordnungsrahmen dienen drei Bedürfnisse aus der Selbstbestimmungstheorie, die eine Tätigkeit aus sich heraus tragen können [12]. Kompetenz bezeichnet das Erleben, besser zu werden. Das setzt voraus, dass Fortschritt abgestuft sichtbar ist und ein Fehlschlag eine benennbare Ursache hat. Autonomie bezeichnet den Spielraum, eigene Entscheidungen zu treffen, bis hin zur Wahl und zum Bau einer eigenen Aufgabe. Soziale Eingebundenheit bezeichnet den Bezug zu anderen, sei es über einen Vergleich, einen sichtbaren Namen oder geteilte Inhalte. Das Raster macht die Verteilung der umgesetzten Elemente sichtbar und zeigt, ob ein Entwurf alle drei Bedürfnisse adressiert.

4.4 Passung von Anforderung und Fähigkeit

Eine Aufgabe trägt dann, wenn ihre Anforderung und die Fähigkeit der Lösenden in einem engen Verhältnis stehen, das der Flow-Theorie als Flow-Kanal beschrieben wird [13]. Liegt die Anforderung darüber, entsteht Überforderung. Liegt sie darunter,

entsteht Langeweile. Für eine Lernanwendung mit fester Reihenfolge folgt daraus, dass die Steigerung zwischen zwei Leveln weder zu klein noch zu groß sein darf und dass erkennbar sein sollte, was ein Level verlangt, bevor es geöffnet wird.

4.5 Festlegungen für jedes Spielelement

Aus dem Zweck der Anwendung ergeben sich zwei Festlegungen, die für jedes eingebaute Spielelement gelten. Die gemessenen Größen sind Eigenschaften der Lösung und nicht des Verhaltens, das zu ihr geführt hat. Eine Wertung, die belohnt, wer am meisten Zeit aufwendet oder am schnellsten tippt, erfasst nicht den vermittelten Inhalt, während eine Wertung nach der Zahl der Schritte das Verständnis des Levels erfasst. Zudem steuert keine Belohnung den Zugang zu Lerninhalten. Jedes Level bleibt ohne Konto, ohne Bestenliste und ohne Serverbetrieb vollständig spielbar. Ein Spielelement darf den Reiz einer Aufgabe erhöhen, ihn aber nicht ersetzen, und es darf niemanden ausschließen.

5 Methodik

5.1 Überblick

Die Änderungen im Betrachtungszeitraum lassen sich in sechs Arbeitssträngen zusammenfassen, die aufeinander aufbauen. Der erweiterte Handlungsraum ist Voraussetzung für Level, die mehr als einen Weg verlangen, und die erhobenen Kennzahlen sind Voraussetzung für eine sortierbare Bestenliste. Der Handlungsraum der Drohne wurde um die Befehle Schieben, Aufnehmen, Abliefern und Vorausschauen erweitert, wozu fünf neue Blocktypen und Lösungsbedingungen im Levelformat kamen. Darauf aufbauend wuchs die Aufgabenkurve von sechs auf dreizehn Level bei überarbeiteter Einstiegsreihe und Schlagworten als Vorschau. Die Laufauswertung erfasst seither Abstürze, geflogene Felder und Codezeilen je Lauf, zeigt diese Werte nach Abschluss an und gibt print-Ausgaben in einem Terminal aus. Auf dieser Grundlage entstand eine Bestenliste als FastAPI-Dienst mit Postgres-Datenbank, Konten, JWT-Anmeldung, Rangfolge und Versuchsverlauf. Parallel dazu wurde ein dreidimensionaler Level-Editor mit Testlauf, YAML-Export und Einreichung über GitHub gebaut. Der sechste Strang fasst Arbeiten am Reibungsabbau zusammen, darunter die Darstellung auf schmalen Bildschirmen, eine eigene Fehlerseite, ein Rückmeldeknopf und eine automatisierte Testabdeckung.

6 Ergebnisse

Die in Abschnitt 5 beschriebenen Arbeitsstränge haben eine zuvor nicht vorhandene, geschlossene Spielschleife hervorgebracht. Vor der Erweiterung endete der Ablauf mit dem Erreichen des Ziels. Seither führt jede Auswertung wieder auf eine mögliche nächste Handlung, sei es das nächste Level, der Vergleich in der Bestenliste, die Verbesserung der eigenen Lösung oder der Bau eines eigenen Levels im Editor. Tab. 1 fasst den quantitativen Zuwachs zwischen dem Ausgangsstand v1.0.0 und dem Ende des Betrachtungszeitraums zusammen.

Kennzahl	v1.0.0	28.09.2026
Spielbare Level	6	13
Öffentliche Drohnen-Methoden	10	14
Blocktypen	9	14

Tab. 1: Wachstum der zentralen Kennzahlen zwischen dem Ausgangsstand v1.0.0 und dem Ende des Betrachtungszeitraums.

Hinzu kommt eine automatisierte Testabdeckung von 118 Testfunktionen, die zuvor nicht bestand.

6.1 Erweiterter Handlungsraum und neue Bausteine

6.1.1 Neue Befehle

Der erste Eingriff betraf die Python-Schnittstelle. Solange die Drohne nur fliegen und sich drehen konnte, bestand jedes Ziel darin, eine Position zu erreichen, und die Level unterschieden sich allein in der Geometrie des Weges. Vier neue Befehle veränderten oder prüften seither den Zustand der Welt. Der Befehl `drone.push()` schiebt einen als schiebbar gekennzeichneten Block um ein Feld weiter und lässt die Drohne nachrücken. Ist das Feld dahinter belegt, bleibt die Aktion folgenlos und nennt den Grund. Die Befehle `drone.pickup()` und `drone.deliver()` nehmen ein Paket auf und legen es auf einer Ablagefläche wieder ab, wobei beide melden, wenn nichts aufzunehmen oder nichts abzulegen ist. Der Befehl `drone.scan(n)` liefert die Block-Kennungen der nächsten n Felder in Blickrichtung und bricht am ersten blockierenden Feld ab. Ohne Argument gibt er stattdessen eine einzelne Kennung zurück.

Der Befehl `scan` ersetzt die frühere Abfrage `is_path_blocked`, die nur einen Wahrheitswert lieferte. Das ist eine der Stellen, an denen die Arbeit über den Umfang der Lehrveranstaltung hinausgeht: Der Unterschied wirkt sich auf die Aufgabengestaltung aus, da eine Zeichenkette oder eine Liste dazu zwingt, das Ergebnis zu vergleichen und in eine Bedingung einzusetzen, während ein Wahrheitswert unmittelbar in eine `if`-Anweisung führt. Damit lassen sich Level bauen, in denen die Drohne unterscheiden muss, worauf sie trifft, und nicht nur, ob sie auf etwas trifft.

Insgesamt wuchs die öffentliche Schnittstelle von zehn auf vierzehn Methoden und die Blockregistrierung von neun auf vierzehn Typen. Neu sind Münze, Kiste, Paket, Ablagefläche und Zielmarkierung. Abb. 1 zeigt ein Level, das drei dieser Bausteine gleichzeitig verwendet.

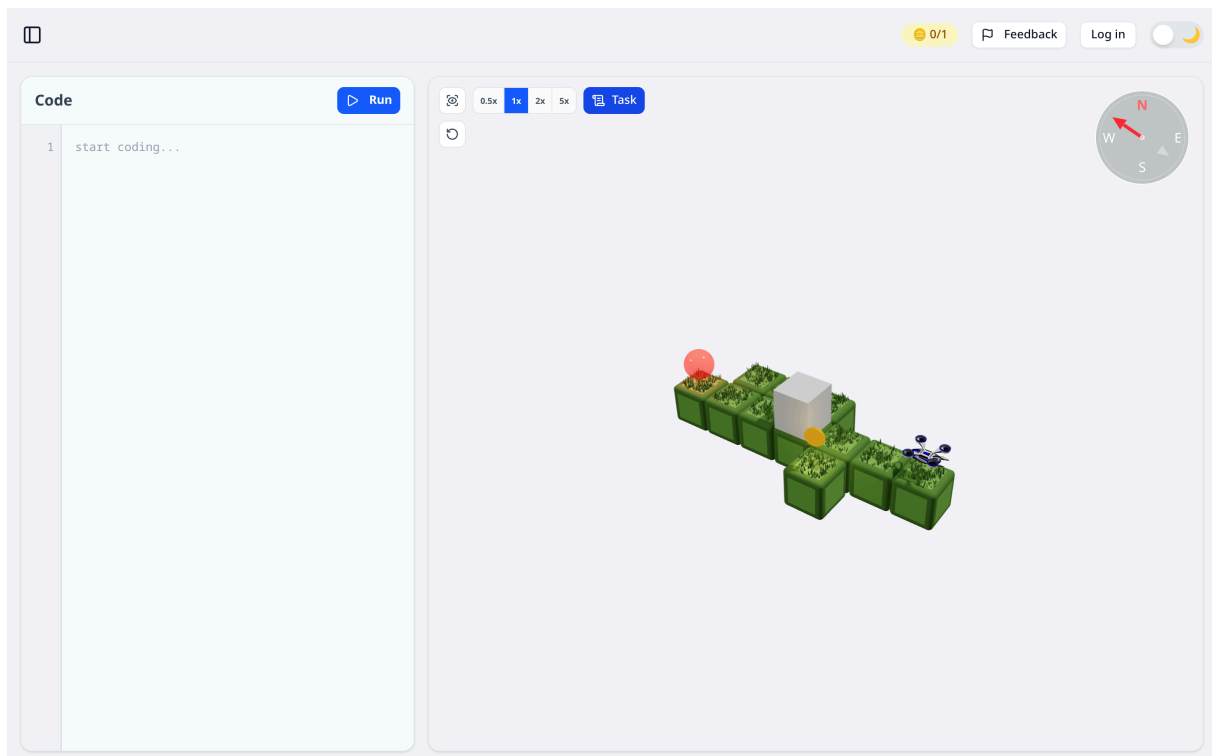


Abb. 1: Das Level *Push 'n Collect* vor dem ersten Lauf. Die Kiste versperrt den direkten Weg, die Münze liegt auf einem Seitenpfad, und der Zähler in der Kopfzeile hält den Stand der Lösungsbedingung fest.

6.1.2 Lösungsbedingungen

Damit die neuen Handlungen Bedeutung bekommen, wuchs das Levelformat mit. Ein Level kann nun neben dem Erreichen des Portals verlangen, dass eine bestimmte Zahl Münzen eingesammelt, ein Paket abgeliefert oder eine Kiste auf ein markiertes Feld geschoben wurde. Die Prüfung liegt im Levelmodell und läuft ab, sobald die Drohne das Portal betritt. Ist eine Bedingung noch offen, sendet die Python-Seite eine eigene Nachricht vom Typ `hint`, deren Text die offenen Bedingungen benennt, sodass Lernende unmittelbar erfahren, warum ein scheinbar erreichtes Ziel noch nicht zählt.

6.2 Aufgabenkurve

Der Levelbestand stieg von sechs auf dreizehn. Ein Teil davon entstand aus einer Überarbeitung der Einstiegsreihe, bei der die ersten Level neu geordnet und ihre Beschreibungen gestrafft wurden. Die neuen Level greifen die erweiterten Befehle auf, sodass jeder Schritt der Reihe entweder ein Programmierkonzept oder eine Drohnenfähigkeit einführt, aber möglichst nicht beides gleichzeitig.

Jedes Level trägt Schlagworte, die in der Übersicht angezeigt werden. Sie nennen eine grobe Schwierigkeit und das behandelte Konzept, etwa Schleifen, Funktionen oder Bedingungen. Diese Vorschau setzt die Passung zwischen Anforderung und Fähigkeit um, weil die Einordnung sichtbar wird, bevor ein Level geöffnet wird. Gesperrte Level bleiben in der Übersicht sichtbar und tragen ein Schloss. Das ist eine bewusste Entscheidung gegen das Ausblenden, da ein sichtbares, aber verschlossenes Ziel als Anreiz wirken kann, ein ausgeblendetes hingegen nicht.

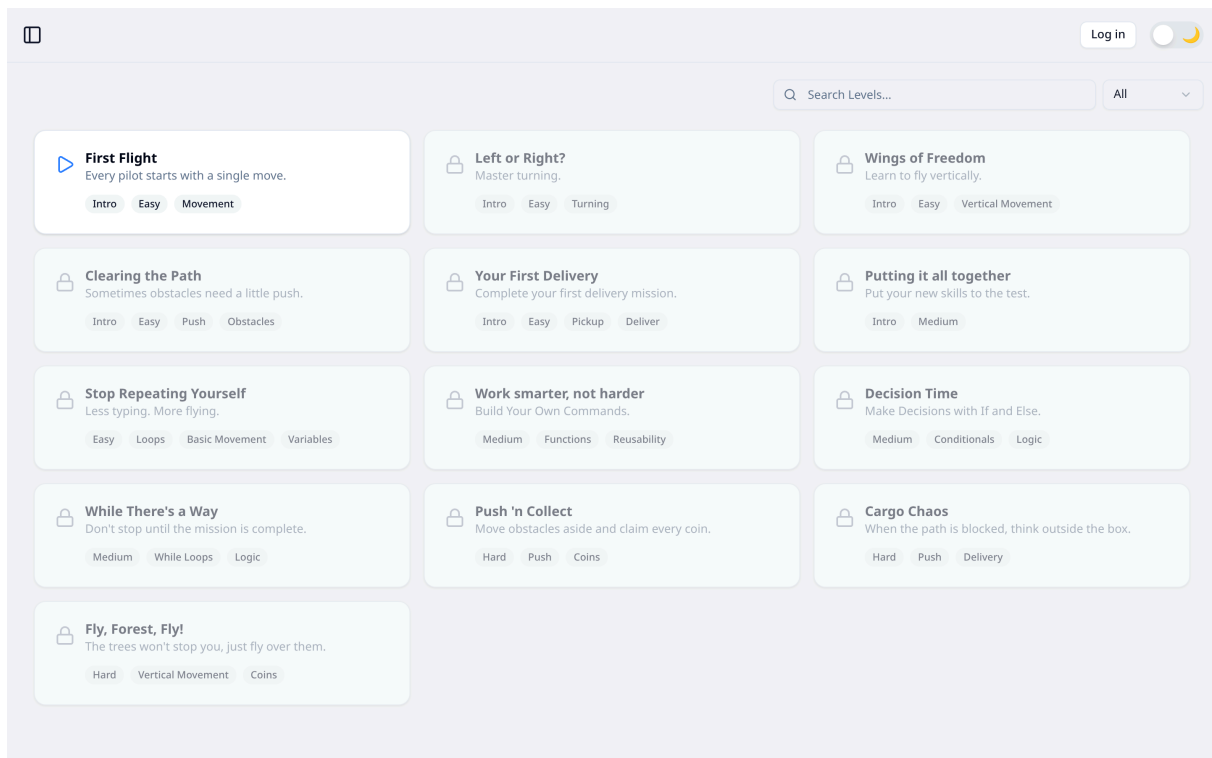


Abb. 2: Die Levelübersicht mit dreizehn Einträgen. Freigeschaltete Level tragen ein Abspielsymbol, gesperrte ein Schloss. Die Schlagworte kündigen Schwierigkeit und behandeltes Konzept an.

6.3 Auswertung eines Laufs

6.3.1 Kennzahlen

Die Python-Seite führt zwei Zähler mit. `distance` steigt bei jedem erfolgreichen Ortswechsel, `crash_count` bei jeder Kollision. Beide werden über `get_stats()` an den Hauptthread gereicht. Die dritte Kennzahl, die Zahl der Codezeilen, ermittelt die Oberfläche selbst aus dem eingegebenen Text.

Diese Auswahl folgt der ersten der beiden Festlegungen aus Abschnitt 4.5. Alle drei Größen beschreiben das Programm und nicht die Person. Die Zahl der Felder eines Weges, die Häufigkeit einer Kollision und die Kompaktheit der Formulierung sind Eigenschaften des Algorithmus. Die Tippgeschwindigkeit gehört nicht dazu.

6.3.2 Abschlussanzeige

Beim Erreichen des Ziels erscheint eine Anzeige mit der benötigten Zeit und den drei Kennzahlen, ergänzt um einen kurzen Kommentar, der sich nach der Zahl der Abstürze richtet. Der Kommentar unterscheidet einen Lauf ohne Kollision von einem mit mehreren, ist bewusst knapp gehalten und ordnet die Zahl ein.

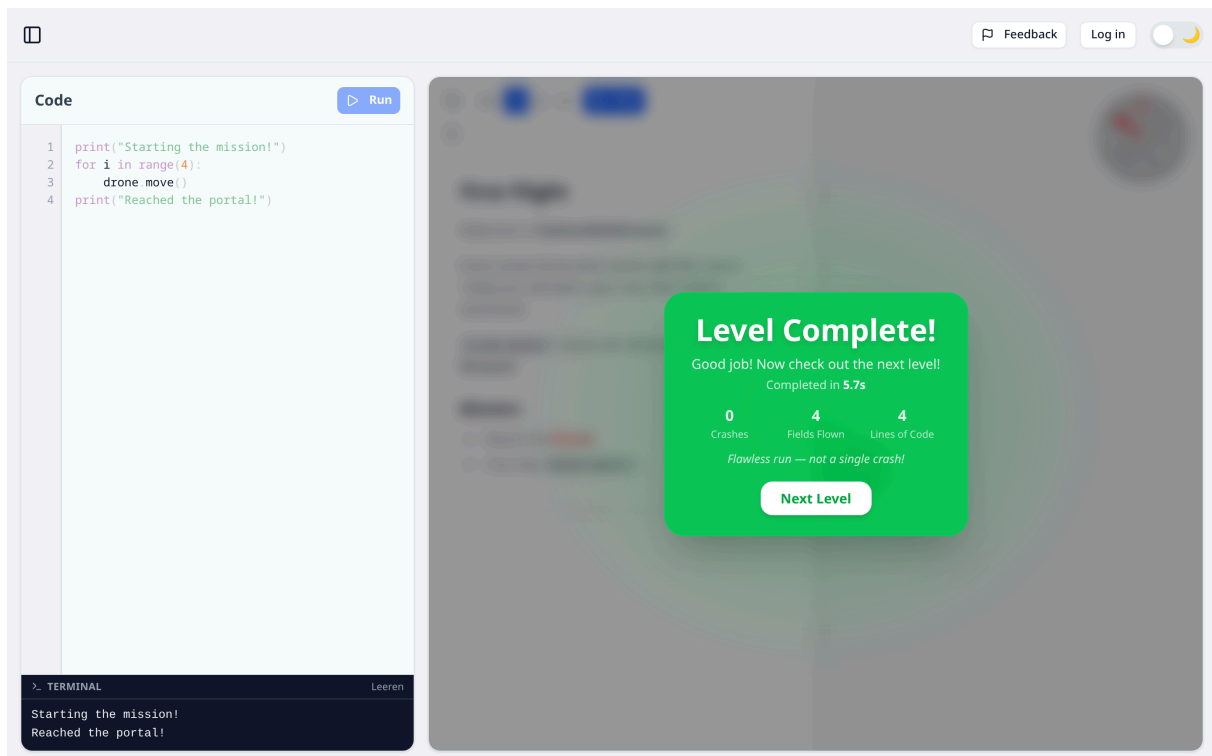


Abb. 3: Die Abschlussanzeige nach einem gelösten Level mit Zeit, Abstürzen, geflogenen Feldern und Codezeilen. Unten links zeigt das Terminal die Ausgaben der beiden `print`-Aufrufe.

6.3.3 Terminal

Ursprünglich lief die Ausgabe von `print` ausschließlich in die Entwicklerkonsole des Browsers. Für die Zielgruppe war sie damit praktisch nicht vorhanden, obwohl `print` das erste Werkzeug ist, mit dem Anfänger den Zustand eines Programms untersuchen. Der Worker leitet die Ausgabe jetzt zusätzlich als eigene Nachricht an den Hauptthread weiter, wo sie unter dem Editor in einem Terminalbereich erscheint. Denselben Ablauf nutzen auch die Hinweistexte der Drohnenbefehle, etwa wenn `pickup` ins Leere greift.

Der Beitrag zur Gamifizierung besteht hier nicht in einem Spielelement, sondern in einer kürzeren Rückmeldeschleife, da ein Fehler mit sichtbarer Ursache eher zu einem weiteren Versuch führt als ein Fehler ohne erkennbare Ursache.

6.4 Bestenliste

6.4.1 Architektur

Die Bestenliste ist der einzige Teil des Projekts, der eine serverseitige Komponente verlangt und damit der größte einzelne Sprung über den Umfang der Lehrveranstaltung hinaus. Sie besteht aus einem FastAPI-Dienst [6] und einer Postgres-Datenbank [8], die zusammen mit dem Frontend über Docker Compose gestartet werden. Der Datenbankzugriff läuft asynchron über SQLAlchemy [7]. Kennwörter werden mit Argon2 gehasht, die Anmeldung liefert ein JWT, und die Anmelde- und Registrierungsrouten sind ratenbegrenzt. Zugelassene Ursprünge stehen in einer Liste in der Umgebungsconfiguration.

Der Dienst führt drei Tabellen. `users` hält die Konten, wobei die Eindeutigkeit des Namens über einen funktionalen Index auf der kleingeschriebenen Fassung erzwungen

wird, sodass sich zwei Konten nicht allein durch Groß- und Kleinschreibung unterscheiden können. `scores` hält je Konto und Level genau einen Eintrag, nämlich die erste Lösung. `attempts` hält jeden einzelnen Versuch. Abb. 4 zeigt die drei Tabellen mit ihren Spalten und der Beziehung zwischen ihnen.

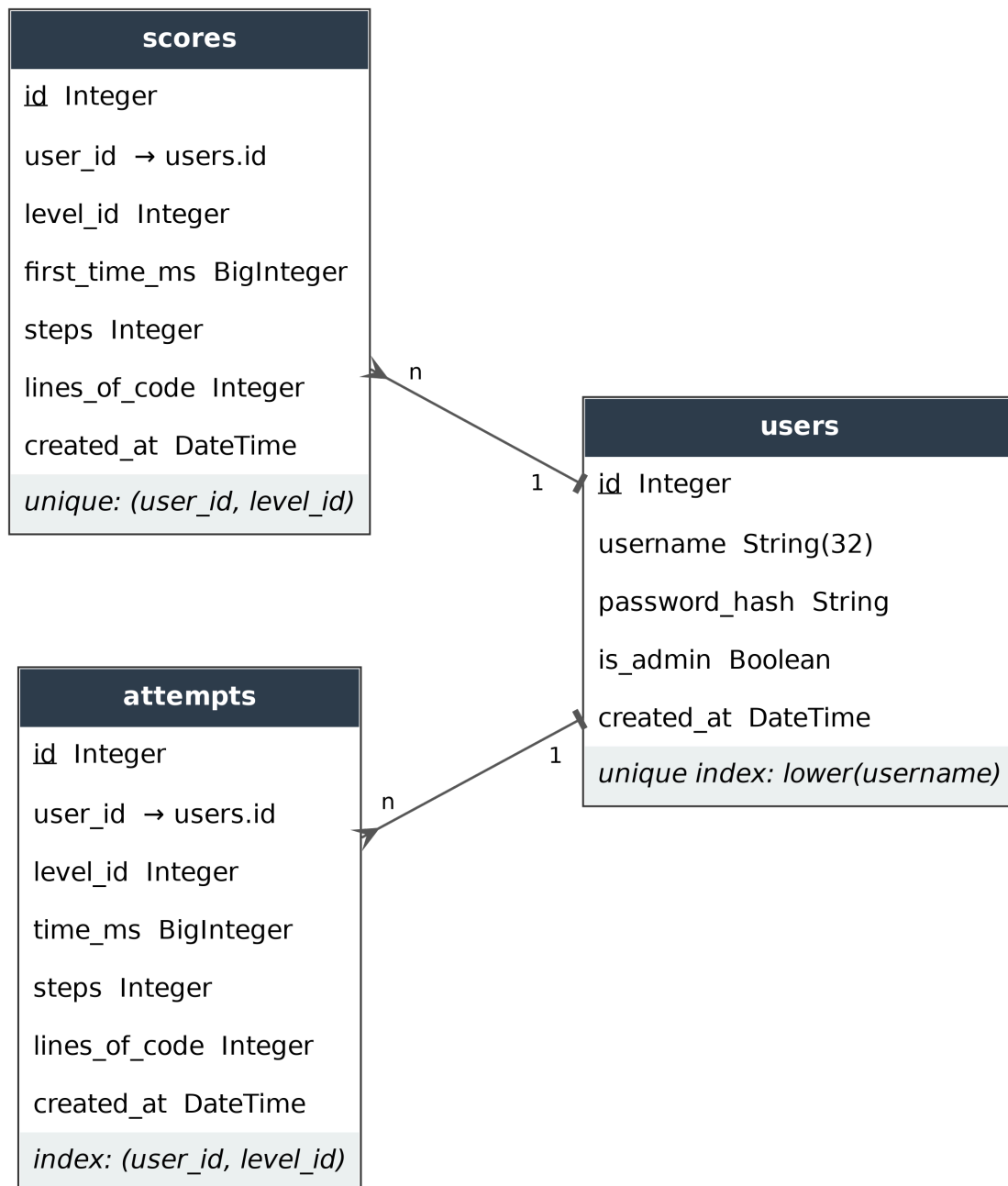


Abb. 4: Datenbankschema der Bestenliste. Jedes Konto in `users` kann beliebig viele Zeilen in `scores` und `attempts` besitzen, jede dieser Zeilen gehört genau einem Konto. `scores` erzwingt zusätzlich Eindeutigkeit je Konto und Level, `attempts` nicht.

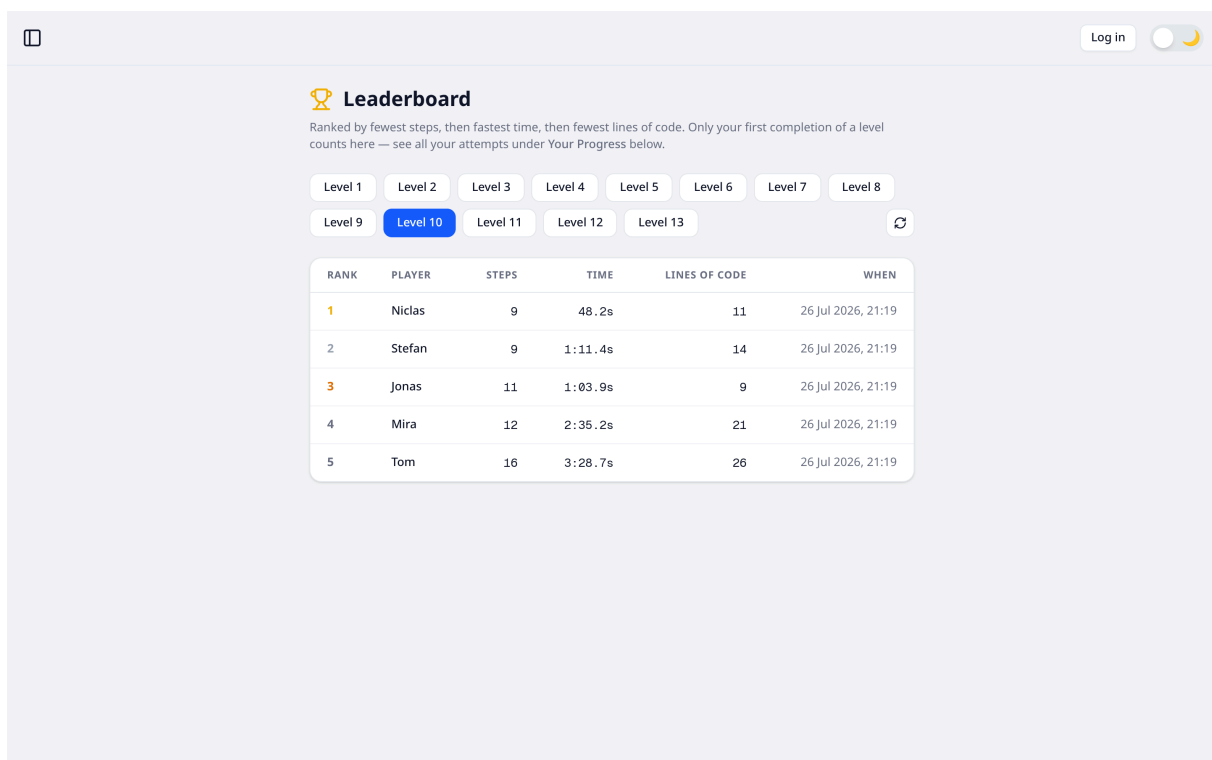
6.4.2 Was gewertet wird

Die Rangfolge ergibt sich zuerst aus der Zahl der Schritte, dann aus der benötigten Zeit und zuletzt aus der Zahl der Codezeilen. Diese Reihenfolge ist die zentrale Entscheidung dieses Strangs. Stünde die Zeit an erster Stelle, würde die Bestenliste vor allem die Tippgeschwindigkeit und die Vorkenntnis der Lösung abbilden. Die Schrittzahl dagegen

ist eine Eigenschaft des gefundenen Weges und lässt sich nur verbessern, indem das Level besser verstanden wird.

Gewertet wird die erste Lösung eines Levels. So bleibt die Liste ein Vergleich der Lösungsqualität und wird nicht zu einem Vergleich der aufgewendeten Zeit. Ergänzend hält die Tabelle `attempts` den vollständigen Verlauf, den Angemeldete unter der eigenen Fortschrittsanzeige einsehen können. Der Vergleich mit anderen bezieht sich damit auf die Erstlösung, während der Verlauf den Vergleich mit der eigenen früheren Leistung dauerhaft ermöglicht.

Die Zeitmessung beginnt beim Öffnen des Levels und endet bei der ersten erfolgreichen Lösung. Sie läuft als Wanduhr im Hauptthread und ist damit unabhängig davon, mit welcher Geschwindigkeit die Animation abgespielt wird oder wie lange der Python-Interpreter für die Ausführung braucht.



RANK	PLAYER	STEPS	TIME	LINES OF CODE	WHEN
1	Niclas	9	48.2s	11	26 Jul 2026, 21:19
2	Stefan	9	1:11.4s	14	26 Jul 2026, 21:19
3	Jonas	11	1:03.9s	9	26 Jul 2026, 21:19
4	Mira	12	2:35.2s	21	26 Jul 2026, 21:19
5	Tom	16	3:28.7s	26	26 Jul 2026, 21:19

Abb. 5: Die Bestenliste einer laufenden Instanz. Die Level lassen sich einzeln auswählen, die Spaltenköpfe sind sortierbar. Die Reihenfolge folgt zuerst der Schrittzahl, wie der Vergleich der ersten drei Ränge zeigt.

6.4.3 Betrieb ohne Backend

Die Anwendung bleibt weiterhin ohne Server benutzbar, weil sie als statischer Export auf GitHub Pages liegt. Der API-Client liest dazu eine einzige Umgebungsvariable. Ist sie nicht gesetzt, liefert jeder Aufruf `null` zurück, und sämtliche Aufrufe der Bestenliste laufen ins Leere, ohne einen Fehler zu erzeugen. Das Spiel funktioniert dann vollständig, nur ohne Vergleich.

Anmeldung ist an keiner Stelle Voraussetzung für das Spielen. Der Anmeldedialog bietet neben Registrierung und Anmeldung ausdrücklich einen Gastmodus an. Damit ist die zweite Festlegung eingehalten, da kein Lerninhalt an einer Belohnungsmechanik hängt.

6.5 Level-Editor

6.5.1 Aufbau

Der Editor ist der umfangreichste Einzelbeitrag des Betrachtungszeitraums und wurde in mehreren Durchgängen ausgebaut. Er läuft vollständig im Browser und arbeitet auf derselben Datenstruktur wie das Spiel selbst.

Die Oberfläche trennt zwei Modi. Im Bearbeitungsmodus platziert oder löscht ein Klick einen Block auf der aktiven Ebene, im Kameramodus dreht und zoomt derselbe Klick die Ansicht. Da beim Bauen ständig zwischen beidem gewechselt wird, lässt sich der Kameramodus zusätzlich durch Halten der Alt-Taste vorübergehend aktivieren, das Löschen entsprechend über die X-Taste. Neben Setzen und Löschen gibt es ein Werkzeug für den Startpunkt der Drohne.

Die Ebenen können einzeln selektiert werden. Sichtbar bleiben dabei alle Ebenen, die aktive wird durch ein Gitter hervorgehoben, sodass die Tiefe beim Bauen erhalten bleibt. Ein eigenes Formular legt die Ausdehnung in allen drei Achsen fest, ein weiteres Titel, Beschreibung und Schlagworte.

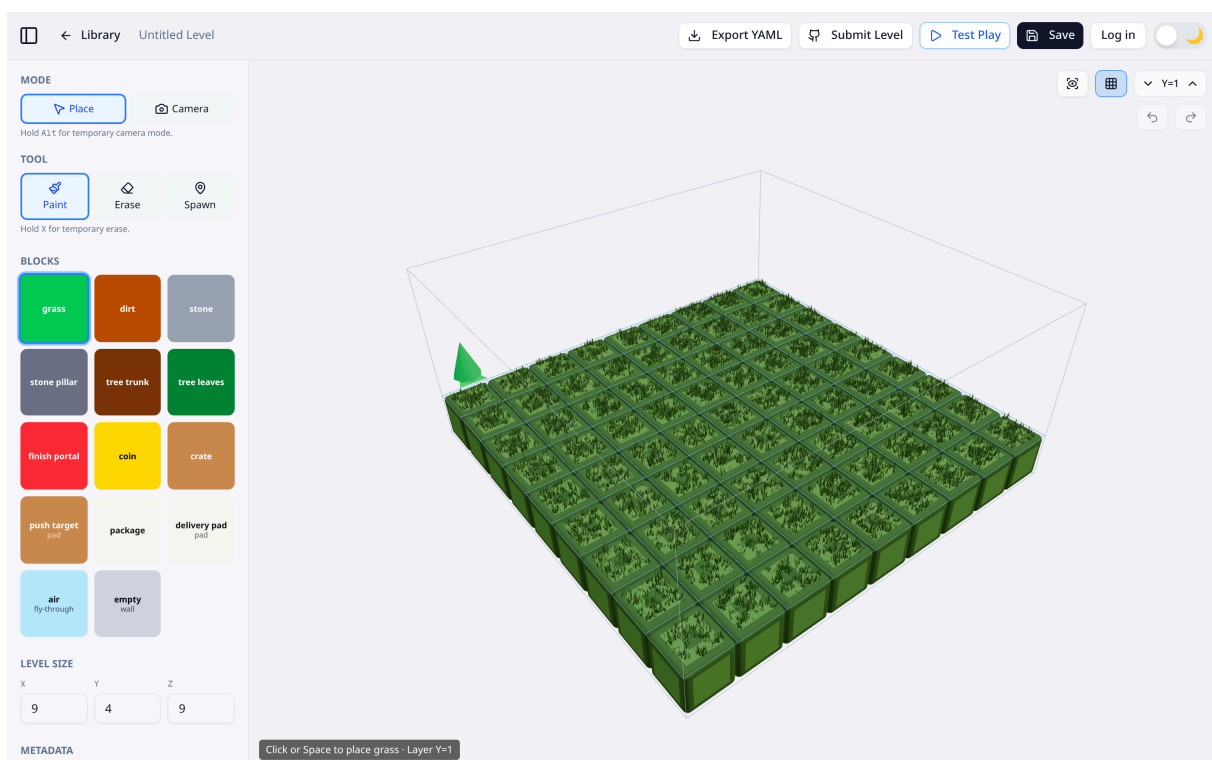


Abb. 6: Der Level-Editor mit einem geladenen Level. Links Modus, Werkzeuge, Blockpalette und Größenangaben, oben rechts der Ebenenwähler und die Rücknahme, oben die Aktionen Export, Einreichung, Testlauf und Speichern.

6.5.2 Vom Entwurf zum Level

Drei Wege führen aus dem Editor heraus. Der Testlauf öffnet das gebaute Level in einem Dialog mit vollständiger Spielumgebung, also eigenem Editor, eigener Pyodide-Instanz [4] und eigener Szene. Der Dialog baut diese Umgebung erst beim Öffnen auf und wirft sie beim Schließen wieder weg, damit nicht dauerhaft ein zweiter Interpreter mitläuft. Der Export schreibt eine YAML-Datei in den Download-Ordner. Die Einreichung öffnet

ein vorbereitetes GitHub-Issue, in dessen Formular das erzeugte YAML bereits eingetragen ist. Ein selbst gebauter Editor mit isoliertem Testlauf ging deutlich über die im Modul vermittelten Inhalte hinaus und war zugleich die größte einzelne Herausforderung des Betrachtungszeitraums, weil Editor und Spiel dieselbe Datenstruktur ohne Umweg über das Dateisystem teilen mussten.

Ursprünglich mussten Autoren die Lösungsbedingungen eines Levels von Hand angeben, also etwa eintragen, dass drei Münzen einzusammeln sind. Seitdem leitet der Editor die Bedingungen aus dem gebauten Level ab. Wer drei Münzen platziert, baut ein Level mit drei einzusammelnden Münzen. Damit können Geometrie und Bedingung nicht mehr auseinanderlaufen, und der Bauvorgang wird um einen Schritt kürzer.

6.6 Reibungsabbau

Der sechste Strang enthält Arbeiten, die für sich genommen keine Spielelemente sind, aber Voraussetzung dafür, dass die anderen fünf wirken.

Die Darstellung auf schmalen Bildschirmen wurde überarbeitet. Betroffen waren unter anderem der Ebenenwähler des Editors, der außerhalb des sichtbaren Bereichs lag, ein Mausextraneum, das nie gebunden wurde, und eine überfüllte Kopfzeile. Da ein erheblicher Teil der Zugriffe von Mobilgeräten erfolgt, beeinflusst diese Ebene, ob die erste Aufgabe überhaupt erreicht wird.

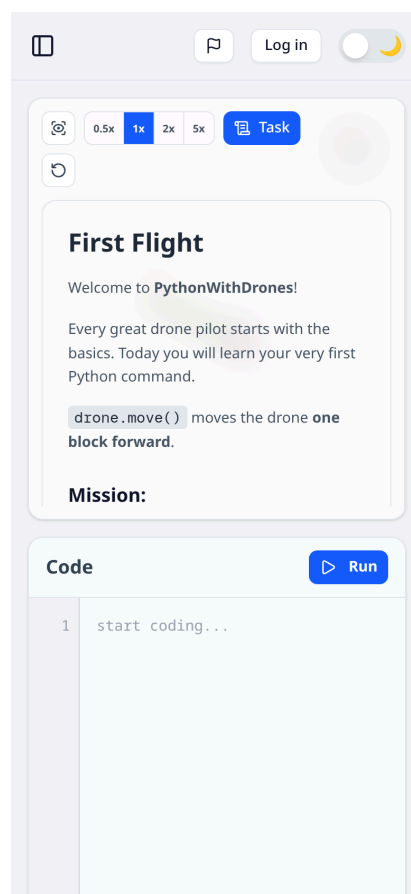


Abb. 7: Die Levelansicht auf einem schmalen Bildschirm. Szene und Editor stehen untereinander statt nebeneinander, der Aufgabentext lässt sich über die Schaltfläche Task ein- und ausblenden.

Ein zweiter Punkt betrifft ungültige Adressen. Seit Juli existiert eine eigene Fehlerseite mit animierter Drohne, und die Routenkonfiguration weist unbekannte Parameter sauber ab. Ein Aufruf von `/level/99` landet damit auf der gestalteten Seite des Spiels statt auf einer Fehlermeldung des Servers. Das ist kein Spielelement, verhindert aber, dass eine falsche Adresse wie ein Defekt der Anwendung wirkt.

Schließlich entstand eine automatisierte Testabdeckung, die im Umfang der Lehrveranstaltung nicht angelegt war. 118 Testfunktionen prüfen die Spiellogik in `game.py` und `model.py` sowie den Bestenlisten-Dienst. Die Spieltests täuschen dazu das Brückenmodul zur JavaScript-Seite vor, sodass der unveränderte Quelltext unter normalem CPython läuft, eine Technik, die im Modul nicht behandelt wurde. Die Dienst-Tests sprechen die Anwendung im selben Prozess an und arbeiten je Test gegen eine wegwerfbare SQLite-Datenbank statt gegen Postgres. Damit ist die Spiellogik, die im Browser sonst nur schwer zu prüfen wäre, in einer gewöhnlichen Testumgebung abgesichert.

7 Diskussion

7.1 Zuordnung zu den Grundbedürfnissen

Die umgesetzten Elemente lassen sich den drei Grundbedürfnissen zuordnen. Kennzahlen, Abschlussanzeige, Versuchsverlauf, gestufte Levelreihe, Terminal und die Hinweise bei offenen Lösungsbedingungen machen Fortschritt abgestuft statt binär sichtbar und geben einem Fehlschlag eine benennbare Ursache, das bedient die Kompetenz. Der Level-Editor adressiert das Bedürfnis nach Autonomie, das von Punkten und Bestenliste nicht bedient wird. Er verschiebt die Rolle vom Lösen vorgegebener Aufgaben zum Stellen eigener, und weder die freie Levelwahl noch der Gastmodus setzen dafür ein Konto voraus, sodass kein Lerninhalt an eine Belohnung gebunden ist. Für den Lerneffekt ist das insofern bedeutsam, als das Bauen eines lösbaren Levels verlangt, die Regeln der Simulation vollständig verstanden zu haben. Gleichzeitig ist der Editor eine Inhaltsquelle, denn der Aufwand für ein neues Level lag zuvor bei handgeschriebenem YAML und mehreren synchron zu haltenden Stellen im Quelltext und liegt nun bei einem Formular und einer Einreichung per Klick. Sozial eingebunden wird, wer über die Bestenliste je Level, den sichtbaren Kontonamen oder die Einreichung eigener Level in Bezug zu anderen tritt. Der zuvor rein lokale Fortschritt bekommt damit erstmals einen Rahmen außerhalb des eigenen Browsers.

Am weitesten ausgebaut ist der Kompetenzstrang, weil er auf jeden einzelnen Lauf unmittelbar zurückwirkt. Die soziale Ebene ist mit Bestenliste und eingereichten Levels die jüngste der drei und bietet für die weiteren Schritte den größten Spielraum.

7.2 Einschränkungen und Reichweite der Aussagen

Die Wirkungsaussagen dieser Arbeit sind Entwurfsargumente, die sich aus dem gebauten System und aus den vorangestellten Festlegungen ergeben. Eine Nutzerstudie war im Zeitrahmen des Moduls nicht vorgesehen. Eine Erhebung mit tatsächlichen Programmieranfängern ist der geeignete Weg, um die hier begründeten Wirkungen zu prüfen, weil die Zielgruppe der Anwendung genau diese Gruppe ist. Da keine empirische

Erhebung stattfand, liegen auch keine quantitativen Verhaltensdaten vor, die sich auf Plausibilität prüfen ließen. Diese Prüfung ist damit selbst eine Einschränkung der vorliegenden Arbeit und wird in Abschnitt 8 als nächster Schritt vorgeschlagen.

Darüber hinaus bestehen drei konkrete Einschränkungen:

- **Fehlende Funktionalität:** Abzeichen als dritter Standard-Baustein der Gamifizierung sind zurückgestellt, und eine serverseitige Plausibilitätsprüfung der eingereichten Kennzahlen existiert nicht.
- **Einschränkungen der verwendeten Mittel:** Im Editor gebaute Level verlassen den Browser nur über einen YAML-Export oder ein vorbereitetes GitHub-Issue. Ohne GitHub-Konto lässt sich ein eigenes Level damit nicht veröffentlichen.
- **Einschränkungen der Qualität:** Die Bestenliste vertraut den vom Client übermittelten Kennzahlen vollständig. In einem offen zugänglichen Wettbewerb ließe sich dieses Vertrauensmodell ausnutzen.

7.3 Bewusst getroffene Festlegungen

Mehrere Entscheidungen im Entwurf hatten Alternativen, die aus guten Gründen nicht gewählt wurden. Sie werden hier zusammengefasst, weil sie den Rahmen abstecken, in dem das Ergebnis zu lesen ist.

Die Kennzahlen werden im Browser gezählt und unverändert vom Dienst übernommen. Die Ausführung bleibt damit vollständig auf der Clientseite, und die Bestenliste braucht dafür keine zusätzliche Rechenlast. Für den Einsatz in einer Lehrveranstaltung reicht dieses Vertrauensmodell aus. In einem offen zugänglichen Wettbewerb müsste stattdessen der Dienst die eingereichte Lösung selbst nachrechnen.

Die Bestenliste bringt eine serverseitige Komponente in ein Projekt, das ursprünglich ohne sie auskam. Diese Abhängigkeit ist über eine Umgebungsvariable abgeschwächt, die den Dienst bei Bedarf still abschaltet, statt ihn zur Voraussetzung zu machen. Dieselbe Auslieferung bleibt so mit und ohne Dienst vollständig spielbar, und die Anwendung gewinnt den Vergleich, ohne ihre Unabhängigkeit von einem Betrieb aufzugeben. Der ausdrücklich angebotene Gastmodus hält zusätzlich die zweite Festlegung aus Abschnitt 4.5 ein, wonach kein Lerninhalt an eine Belohnungsmechanik gebunden sein darf.

Im Editor gebaute Level liegen im `localStorage` des jeweiligen Browsers und verlassen ihn nur über eine YAML-Datei oder ein vorbereitetes GitHub-Issue. Für die Übergabe an das Projektteam genügt das, und der Editor bleibt dabei frei von jeder Kontopflicht.

7.4 Wertungslogik und Motivation

Eine eingeführte Belohnung kann die ursprüngliche Motivation überlagern, wenn sie auf etwas anderes zeigt als auf das Lernziel [12]. Dies war beim Entwurf der Wertung der maßgebliche Prüfpunkt. Zwei Entscheidungen begrenzen dieses Risiko. Die Rangfolge nach Schritten belohnt das Verständnis des Levels, das ohnehin Lernziel ist, und lässt sich nicht durch bloßen Fleiß ersetzen. Die Beschränkung auf die erste Lösung verhindert, dass die Liste allein durch Wiederholung verbessert werden kann. Die Belohnung zeigt damit auf dieselbe Fähigkeit, die die Anwendung vermitteln soll.

8 Schlussfolgerung

8.1 Zusammenfassung

Zu Beginn des Moduls konnte die Anwendung Python im Browser ausführen und eine Drohne animieren, der Ablauf endete jedoch mit dem Erreichen des Ziels. Nach der Erweiterung besteht eine geschlossene Schleife, in der ein Level eine Aufgabe stellt, die Simulation die Folgen des geschriebenen Programms zeigt, eine Auswertung die Lösung nach Schritten, Abstürzen und Codezeilen bewertet, eine Bestenliste sie neben die anderer stellt und ein Editor den Bau der nächsten Aufgabe erlaubt.

Der Umfang der Änderungen zeigt sich in den Kennzahlen des Projekts. Die öffentliche Schnittstelle der Drohne wuchs von zehn auf vierzehn Methoden und die Blockregistrierung von neun auf vierzehn Typen, unter anderem durch Schieben, Aufnehmen, Abliefern und Vorausschauen. Der Levelbestand wuchs von sechs auf dreizehn Level mit überarbeiteter Einstiegsreihe und Schlagworten als Schwierigkeits-Vorschau. Jeder Lauf wird seither nach Abstürzen, geflogenen Feldern und Codezeilen bewertet, ergänzt um ein Terminal für `print`-Ausgaben. Hinzu kamen ein FastAPI-Dienst mit Postgres-Datenbank, der Konten, Rangfolge und Versuchsverlauf verwaltet, aber über eine Umgebungsvariable optional bleibt, ein dreidimensionaler Level-Editor mit Testlauf, YAML-Export und Einreichung über GitHub samt automatisch abgeleiteten Lösungsbedingungen, sowie 118 automatisierte Testfunktionen, die die Spiellogik und den Bestenlisten-Dienst außerhalb des Browsers absichern.

Die Auswertung zeigt, dass die umgesetzten Elemente alle drei Grundbedürfnisse erreichen und dabei den beiden vorangestellten Festlegungen folgen. Gemessen wird die Lösung und nicht die Person, und der Zugang zu den Lerninhalten hängt an keiner Belohnung.

8.2 Nächste Schritte

Aus dem erreichten Stand ergeben sich mehrere Ansätze für die Weiterarbeit. Eine Nutzerstudie mit Personen ohne Programmiererfahrung, aufgeteilt in eine Gruppe mit und eine ohne aktivierte Bestenliste, könnte die hier begründeten Wirkungen empirisch prüfen. Die eingereichten Kennzahlen ließen sich durch eine serverseitige Plausibilitätskontrolle absichern, mindestens gegen eine untere Schranke der Schrittzahl je Level. Eine Ablage der im Editor gebauten Level im Backend würde es erlauben, sie ohne GitHub-Konto zu veröffentlichen und von anderen spielen zu lassen. Abzeichen für das erstmalige Lösen eines Levels mit einer Schleife, einer Funktion oder einer Bedingung könnten den bislang zurückgestellten dritten Standardbaustein besetzen, ohne den Wettbewerbsdruck zu erhöhen. Die eingereichten Nutzer-Level ließen sich zusätzlich zu einer rotierenden Auswahl kuratieren, die alle Spielenden für eine begrenzte Zeit gemeinsam spielen, was die soziale Ebene über den reinen Rangvergleich hinaus erweitern würde. Schließlich könnte eine zweite Wertung neben der gewerteten Erstlösung die jeweils beste eingereichte Lösung abbilden, sodass sich auch das Überarbeiten eines Programms in der Rangfolge niederschlägt.

Literatur

- [1] N. Falke, S. Reitemeyer, und J. Pottmeier, „Python with Drones“, Ausarbeitung im Modul Software Design, 2026.
- [2] „PythonWithDrones, Quelltext-Repository“. [Online]. Verfügbar unter: <https://github.com/pottmeier/PythonWithDrones>
- [3] „Next.js Documentation“. [Online]. Verfügbar unter: <https://nextjs.org/docs>
- [4] „Pyodide Documentation“. [Online]. Verfügbar unter: <https://pyodide.org/>
- [5] „React Three Fiber Documentation“. [Online]. Verfügbar unter: <https://r3f.docs.pmnd.rs/>
- [6] „FastAPI Documentation“. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/>
- [7] „SQLAlchemy Documentation“. [Online]. Verfügbar unter: <https://docs.sqlalchemy.org/>
- [8] „PostgreSQL Documentation“. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/>
- [9] S. Deterding, D. Dixon, R. Khaled, und L. Nacke, „From Game Design Elements to Gamefulness: Defining "Gamification"“, in *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*, ACM, 2011, S. 9–15.
- [10] K. Werbach und D. Hunter, *For the Win: How Game Thinking Can Revolutionize Your Business*. Wharton Digital Press, 2012.
- [11] R. Hunicke, M. LeBlanc, und R. Zubek, „MDA: A Formal Approach to Game Design and Game Research“, in *Proceedings of the AAAI Workshop on Challenges in Game AI*, 2004.
- [12] E. L. Deci und R. M. Ryan, „The "What" and "Why" of Goal Pursuits: Human Needs and the Self-Determination of Behavior“, *Psychological Inquiry*, Bd. 11, Nr. 4, S. 227–268, 2000.
- [13] M. Csikszentmihalyi, *Flow: The Psychology of Optimal Experience*. Harper, Row, 1990.